

NativeScript uses reflections to build a list of metadata for native APIs of the platform it's running on, making direct transaction of your JavaScript code to its native equivalents. In simplest terms, the NativeScript runtime acts as a bridge between your JavaScript code and the native APIs.

For example, you can declare a button using the `<Button>` tag in NativeScript. When this code is executed by NativeScript runtime, it will be mapped to the `UIButton` on iOS and to the `android.widget.Button` on an Android phone.

The NativeScript runtime may seem like magic, but it is not — the architecture isn't all that complex. Everything starts with JavaScript virtual machines, as they're what NativeScript uses to execute JavaScript commands. Specifically, NativeScript uses V8 on Android and JavaScript Core on iOS. Because NativeScript uses JavaScript VMs, all the native API-accessing code you write still needs to use JavaScript constructs and syntaxes that V8 and JavaScript Core understand.

Building a demo application using NativeScript

With the theory out of the way, let's get started with a hands-on example. In this demo I'll cover the following topics.

- Setting up the NativeScript development environment
- Building a very simple RSS reader application that covers most coding aspects of NativeScript

Setting up the NativeScript Development environment

1. Install Node.js: NativeScript CLI uses Node.js so you will need it to be installed on your system. Head on to <https://nodejs.org/> and install its latest version.
2. Install the NativeScript CLI using the admin command prompt or terminal with super user privileges, using the following command:

```
npm install nativescript -g
```

3. To build the apps for the platform, you will need a bunch of dependencies like Java, Android SDK, Xcode, etc. Fortunately, the `tns` command provides quick-start scripts for Windows and macOS to automate the installation and configuration of these dependencies. If you are using a Windows machine, type in the following command using a command prompt with administrator privileges:

```
@powershell -NoProfile -ExecutionPolicy Bypass -Command "iex ((new-object net.webclient).DownloadString('https://www.nativescript.org/setup/win'))"
```

On the Mac, you can run the following command with super user privileges:

```
ruby -e "$(curl -fsSL https://www.nativescript.org/setup/mac)"
```

You can refer to the official installation guide at <https://docs.nativescript.org/angular/start/quick-setup>.

Note: To build iOS applications, you need a Mac machine. Later in the article, we will look at how to build your code for iOS using the cloud build environment.

Building a very simple RSS reader application that covers most of the coding aspects of NativeScript

In this sample application, we will create a small RSS reader. The main screen will load the RSS feed from a URL like <https://opensourceforu.com/feed/>, and display the feed and its description on the first screen. On clicking the 'Read more' button, it will load the link of the post in a Web view in the second screen. Please note, to keep the demo simple and concise, some liberties in the coding standards might have been taken. Screen shots of the application we will be building are shown in Figures 1 and 2.



Figure 1: The main screen loading the RSS feed from a URL



Figure 2: The link of the post in a Web view

- A NativeScript project is necessarily a Node project, and there are sample templates available for JavaScript, TypeScript and Vue.js. In this example, we will create a JavaScript project. Run the following command (`RSSReader` is the folder name where all the project files will be created):

```
tns create RSSReader
```

- You can test your project using the following command:

```
tns run android --emulator
```

Please change the environment to iOS if you want it to run on an iPhone, and you can also remove the `--emulator` flag if you have a physical device connected.